DE LA VERSIÓN SECUENCIAL A LA CONCURRENTE

Manual de estudio del TFG

Para Nicole · Sesión del 12 de octubre de 2026

Rama: version-concurrente
Base conservada: correcciones-secuencial

Clases, métodos, hilos, locks, abortos, archivos y pruebas: una explicación del código que hemos construido, paso a paso y con ejemplos.

La idea central
Varias peticiones pueden estar en curso. Solo una transacción está admitida para operar sobre los datos; las que coinciden con ella abortan por colisión temporal.

Documento de aprendizaje. Describe la implementación de esta fase; los niveles de consistencia siguen pendientes.

Índice de lectura

1. Qué hemos hecho hoy y qué debes recordar 4

2. Las palabras básicas, sin dar nada por sabido 5

3. Antes y después: qué sigue igual 6

4. La rama nueva y el mapa de archivos 7

5. La regla temporal de Maite, con ejemplos 8

6. ControlConcurrencia: los dos datos que mantiene 9

7. intentarIniciar: explicación paso a paso 10

8. finalizar y las dos clases de bloqueo 11

9. Transaccion: datos, estados y motivos 12

10. GestorTransacciones: campos, constructores y conexiones 13

11. ejecutarEscritura y ejecutarLectura 14

12. registrarResultado y guardarHistorico 15

13. El servidor: main y la atención de conexiones 16

14. El servidor: procesarPeticion y el cierre 17

15. El cliente: main, Envio y prepararOperaciones 18

16. ejecutarExperimentos: tres firmas y un recorrido 19

17. ResultadoPeticion, ejecutarPeticion y enviarPeticion 20

18. Bibliotecas y expresiones de Java que aparecen 21

19. Mongo, Redis y DatabaseClient 22

20. HistoricoCsv: guardar filas sin mezclarlas 23

21. ResumenExperimentosCsv y sus quince columnas 24

22. Entender las medidas y no sacar conclusiones falsas 25

23. Las pruebas automáticas: cómo comprobamos la regla 26

24. Más pruebas y resultados reales de hoy 27

25. Cómo ejecutar y encontrar los resultados 28

26. Los archivos de esta validación y Excel 29

27. Cómo leer el código para poder explicarlo 30

28. Límites actuales y relación con los TFG de referencia 31

29. Guion de explicación y próximos pasos 32

30. Glosario y fuentes de esta guía 33

1. Qué hemos hecho hoy y qué debes recordar

Partíamos de un programa secuencial que ya funcionaba: el cliente enviaba una petición, esperaba su respuesta y entonces enviaba la siguiente. El servidor también atendía una conexión por completo antes de pasar a otra. El gestor realizaba la lectura o escritura sobre MongoDB 7, MongoDB 8 y Redis, y guardaba información de la transacción.

Hoy hemos creado la rama version-concurrente y hemos permitido que el cliente tenga varias peticiones en curso y que el servidor atienda varias conexiones mediante hilos. Hemos añadido una regla que decide qué transacción puede realizar las operaciones y cuáles deben abortar por coincidir en el tiempo. También hemos adaptado los registros y las pruebas.

La regla de Maite, confirmada durante la conversación, afecta a cualquier solapamiento: también a dos lecturas y a operaciones sobre claves distintas. Por ello, el resultado no es un sistema que deje ejecutar todas las transacciones simultáneamente. Es un sistema que recibe peticiones concurrentes, admite una transacción para operar y rechaza las que intentan comenzar mientras esa sigue activa.

Las cinco ideas fundamentales son estas:

- Un hilo permite que una tarea avance mientras otra está esperando.
- El cliente y el servidor utilizan grupos de hilos; no son el mismo grupo.
- El gestor comparte un único ControlConcurrencia entre las peticiones que atiende ese servidor.
- Una colisión esperada se distingue de un fallo técnico.
- Los resultados se comprueban con pruebas controladas y con ejecuciones reales; una sola ejecución no demuestra una mejora general de rendimiento.

Ya existían los estados INICIADA, FINALIZADA y ABORTADA, el histórico en Mongo y CSV, y el CSV de resúmenes. No los hemos inventado otra vez: los hemos adaptado. La principal clase de producción nueva es ControlConcurrencia. También hay una nueva clase de pruebas, ConcurrenciaTest.

El código quedó guardado localmente en la nueva rama. Al terminar esta sesión de implementación todavía no se había hecho commit ni push de estos cambios concurrentes.

2. Las palabras básicas, sin dar nada por sabido

Clase y objeto. Una clase describe qué datos y acciones tiene algo. Un objeto es una instancia concreta de esa clase. Transaccion es la clase; T1 y T2 serían dos objetos diferentes, con identificadores y estados propios.

Método. Es una acción definida en una clase. ejecutarEscritura es un método del gestor. Recibe una transacción y la información del escenario y realiza el trabajo correspondiente.

Petición y transacción. La petición es el mensaje que viaja desde el cliente: por ejemplo, escribir un valor. El servidor interpreta ese mensaje y crea un objeto Transaccion. No todas las conexiones llegan a crear una transacción: un mensaje mal formado puede rechazarse antes.

Proceso e hilo. El programa del cliente y el del servidor se ejecutan por separado. Dentro de cada uno puede haber varios hilos. Los hilos de un mismo proceso pueden compartir objetos, y esa posibilidad es tanto útil como delicada.

Concurrencia y paralelismo. Concurrencia significa que varias tareas están en curso durante períodos que se solapan. Paralelismo, en sentido estricto, significa que se ejecuta trabajo al mismo instante en distintos recursos de procesamiento. En esta guía usamos “versión concurrente” para describir con precisión el programa; no afirmamos que cada instrucción se ejecute físicamente al mismo tiempo.

Pool de hilos. Es un grupo de trabajadores reutilizables. Si hay cuatro trabajadores y treinta y dos encargos, no se crean necesariamente treinta y dos trabajadores. Algunos encargos esperan a que quede uno libre.

Estado compartido. Es información que pueden consultar o cambiar varios hilos. En nuestro caso, la referencia a la transacción activa es un ejemplo central.

Condición de carrera. Dos tareas pueden tomar una decisión equivocada porque ambas consultan una información antes de que ninguna termine de actualizarla. No significa que corran rápido: significa que el resultado depende de cómo se intercalan sus acciones.

Lock y sección crítica. Un lock controla el acceso a una parte del código que necesita ejecutarse sin interferencias de otros hilos. Esa parte se llama sección crítica. No bloquea automáticamente todas las bases de datos del sistema.

Aborto. Significa que esa transacción no se completó según las reglas. Por sí solo no significa que se hayan deshecho cambios anteriores. Esa distinción será importante durante toda la guía.

3. Antes y después: qué sigue igual

El recorrido general se conserva:

```
Cliente -> Servidor -> Gestor -> Mongo7, Mongo8 y Redis
   ^          |          |
   |_________ respuesta __|
```

El cliente sigue preparando experimentos. El servidor sigue interpretando el mensaje. El gestor sigue coordinando las bases. Mongo y Redis siguen siendo las clases que saben realizar sus operaciones concretas. Esta separación evita que el cliente tenga que conocer las conexiones de las bases.

Antes, el cliente encadenaba “enviar, esperar, enviar, esperar”. Ahora entrega varias tareas a un pool. Antes, el servidor hacía todo el trabajo de una conexión dentro del bucle de escucha. Ahora entrega esa atención a otro hilo y vuelve a escuchar.

No hemos convertido en paralelas las tres llamadas internas de una transacción admitida. Una escritura sigue recorriendo primero Mongo7, después Mongo8 y después Redis. Lo mismo ocurre con una lectura. La concurrencia que hemos añadido está entre las peticiones y su atención, con una regla de exclusión para las operaciones de datos.

El nombre ClienteTransaccionesSimple o ServidorTransaccionesSimple no cambia por sí mismo el comportamiento. Hemos conservado esos nombres para evitar duplicar programas y cambiar los perfiles de Maven sin necesidad. La versión se distingue mediante la rama de Git.

Ejemplo: el servidor puede tener cuatro conexiones abiertas. Una está realizando una lectura admitida; otra acaba de recibir un aborto; otra está guardando su histórico; otra espera poder usar el escritor del CSV. Hay trabajo concurrente, aunque no haya dos transacciones admitidas leyendo o escribiendo datos a la vez.

Esto explica por qué “solo una activa” no equivale a “solo un hilo”. Los hilos también realizan tareas de comunicación y registro fuera del intervalo de operaciones de datos.

4. La rama nueva y el mapa de archivos

Creamos version-concurrente desde correcciones-secuencial. No partimos de main porque necesitábamos los cambios más recientes de la versión secuencial. No fusionamos las ramas ni borramos la anterior.

El comando utilizado fue:

```
git switch -c version-concurrente
```

Crear una rama no es copiar la carpeta del proyecto. Git crea una nueva línea de trabajo que parte del mismo estado. A partir de ahí, los futuros commits pueden evolucionar sin modificar la referencia secuencial. Mientras hay cambios sin commit, conviene no cambiar de rama sin revisar antes git status: los archivos locales sin guardar en un commit no son una copia independiente por rama.

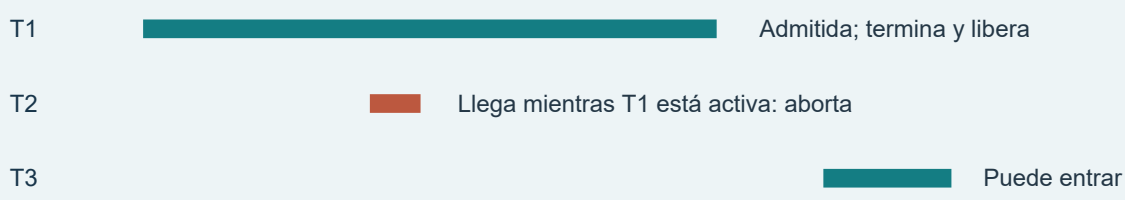
Los archivos Java están en src/main/java/com/nicole/tfg/. Las pruebas están en src/test/java/com/nicole/tfg/.

Archivo	Trabajo de hoy
ControlConcurrencia.java	Nuevo control de admisión y finalización con lock
ClienteTransaccionesSimple.java	Envíos concurrentes, recogida de resultados y clasificación
ServidorTransaccionesSimple.java	Atención concurrente y respuesta de colisión
GestorTransacciones.java	Integración del control antes de tocar los datos
Transaccion.java	Motivo de aborto e inicio ajustable al entrar en el control
HistoricoCsv.java	Escritura sincronizada y nueva columna de motivo
Mongo.java	Motivo de aborto en los documentos del histórico
ResumenExperimentosCsv.java	Concurrencia, colisiones y errores en el resumen
ConcurrenciaTest.java	Cinco pruebas nuevas de concurrencia
README.md	Instrucciones y explicación de la versión actual

Redis.java y DatabaseClient.java no necesitaron cambios para esta fase. El pom.xml conserva las dependencias y perfiles que ya tenía. Algunas pruebas anteriores se adaptaron a los parámetros nuevos del histórico.

5. La regla temporal de Maite, con ejemplos

Ejemplo: quién entra y quién aborta



Si T1 está activa cuando T2 intenta empezar, T2 aborta. No preguntamos si utilizan la misma cuenta ni si son lecturas. Esa regla global es deliberadamente más restrictiva que otros mecanismos de concurrencia.

```
Momento 10: T1 entra en el control y queda admitida.
Momento 12: T2 entra; encuentra T1 activa y aborta.
Momento 15: T1 termina sus operaciones y libera el control.
Momento 16: T3 entra; puede quedar admitida.
```

Los números anteriores son un ejemplo didáctico, no resultados de una ejecución. La idea no depende de que las unidades sean segundos o milisegundos.

¿Qué instante consideramos inicio? El código lo fija dentro de intentarIniciar, bajo el lock. No es el instante de envío del cliente, ni el de aceptación del socket, ni el de creación inicial del objeto. Una petición puede esperar en una cola antes de llegar a ese punto.

¿Qué instante consideramos fin? Se fija dentro de finalizar, después de las operaciones de datos y antes de guardar el histórico. A partir de liberar la posición, otra transacción puede ser admitida, aunque todavía se esté guardando información de la anterior.

¿Cómo sabemos si T1 sigue activa si todavía no conocemos su fin? No buscamos un intervalo completo en Mongo. Conservamos una referencia llamada activa. Mientras no se libere, sabemos que su intervalo de actividad no ha terminado. El lock hace que consultar y modificar esa referencia sea una decisión coordinada.

Los tiempos guardados tienen resolución de milisegundos. Dos sucesos consecutivos pueden llevar la misma marca. Por eso no decidimos una colisión comparando dos números redondeados: la referencia protegida es la que decide. Al analizar después un CSV, la igualdad de marcas no demuestra por sí sola simultaneidad real.

Esta definición concreta importa al hablar con Maite: implementa el solapamiento en la entrada al control del gestor, no el tiempo completo desde que el cliente envía hasta que recibe la respuesta.

6. ControlConcurrencia: los dos datos que mantiene

La clase nueva tiene dos campos:

```
private final ReentrantLock lock = new ReentrantLock();
private Transaccion activa;
```

lock es el objeto que protege las decisiones. final significa que ese campo conservará la referencia al mismo lock. No significa que el lock esté siempre ocupado ni que un objeto final sea automáticamente inmutable.

activa apunta a la transacción admitida. Si vale null, no hay ninguna ocupando la posición. Si contiene T1, otra transacción no puede ser admitida. No es una lista de todas las conexiones abiertas: representa solo el puesto de ejecución de operaciones.

Cada GestorTransacciones tiene un ControlConcurrencia. El servidor crea un único gestor y lo comparte entre los hilos. Si creásemos un gestor distinto para cada conexión, habría controles independientes y varias transacciones podrían entrar sin verse entre sí.

El lock es local a esta aplicación Java. No es un lock distribuido entre varios servidores y no impide que otro programa escriba directamente en Mongo o Redis. Para el experimento actual asumimos un único servidor de transacciones y acceso a los datos a través de él.

La sección protegida es pequeña: consultar el estado, fijar tiempos y cambiar la referencia. Las llamadas a las bases se hacen fuera del lock, pero con activa ocupada. Así, otro hilo puede entrar brevemente en el control, detectar la ocupación y abortar, en lugar de esperar toda la operación.

ReentrantLock es “reentrante” porque un hilo puede adquirir el mismo lock que ya posee, con las liberaciones correspondientes. No aprovechamos esa característica para hacer nada especial aquí. Se utiliza por su forma explícita de adquirir y liberar la protección.

7. intentarIniciar: explicación paso a paso

El método devuelve un boolean: true si la transacción queda admitida; false si colisiona. Hay un caso adicional: si se intenta reutilizar un objeto que ya terminó o ya está activo, lanza una excepción de uso incorrecto.

```
lock.lock();
try {
    // comprobar, fijar inicio y decidir
} finally {
    lock.unlock();
}
```

Paso 1: adquirir el lock. Si otro hilo está actualizando el control, espera a que termine esa sección corta. No es una espera voluntaria hasta que la transacción de datos termine. El orden de acceso al lock no garantiza que gane el cliente que antes comenzó a conectar.

Paso 2: comprobar el objeto. El código exige estado INICIADA y que no sea ya la referencia activa. La expresión activa == tx compara si se trata del mismo objeto, no si tiene el mismo contenido de texto.

Paso 3: fijar el inicio. setTInicial recibe un Timestamp construido a partir del reloj actual. La hora provisional que se creó en el constructor se sustituye por la de entrada al control.

Paso 4: comprobar la ocupación. Si activa no es null, establece ABORTADA, motivo COLISION_TEMPORAL y hora final. Devuelve false. Todavía no se han realizado las operaciones de datos.

Paso 5: admitir si está libre. Si activa es null, asigna activa = tx y devuelve true. La transacción sigue con estado INICIADA mientras ejecuta; no se añadió un estado EJECUTANDO.

Paso 6: liberar el lock. El finally se ejecuta también cuando dentro del try hay un return. Esa es una propiedad importante: el método no deja el lock adquirido simplemente por salir antes.

Sin el lock, T1 y T2 podrían ver activa vacía y asignarse ambas. Con el lock, comprobar y ocupar forman una sola decisión frente a los otros hilos. Aquí “indivisible” se refiere a esa sección de control, no a una transacción distribuida atómica entre las bases.

8. finalizar y las dos clases de bloqueo

finalizar vuelve a adquirir el mismo lock. Comprueba que quien intenta liberar la posición es precisamente la transacción activa. Después establece la hora final y deja activa en null. Por último libera el lock con finally.

El método no decide por sí mismo FINALIZADA o ABORTADA. Eso lo decide el gestor según el resultado de la operación. finalizar se ocupa del tiempo final y de que la posición vuelva a estar disponible.

¿Por qué comprobar activa != tx? Porque una transacción rechazada no debe poder liberar la posición que ocupa T1. Si lo hiciera, T3 podría entrar aunque T1 aún estuviera operando.

En el programa hay dos protecciones con responsabilidades diferentes:

Protección	Recurso protegido	Qué hacen las demás tareas
ReentrantLock del control	La referencia activa y la decisión de admisión	Esperan solo la sección corta; si hay activa, abortan
synchronized del histórico CSV	Cabecera y escritura de una fila	Esperan su turno para guardar la fila

El segundo caso no produce una colisión por sí mismo. Si T1 está escribiendo su histórico y ya liberó el control, T2 puede ser admitida. Sus operaciones y el registro anterior pueden solaparse.

No debemos envolver todo el gestor en un synchronized y considerar el trabajo resuelto. Eso podría convertir las peticiones en una fila de espera sin los abortos por colisión pedidos. Tampoco hay que usar el lock de admisión para escribir cada CSV: son recursos y decisiones diferentes.

Una limitación práctica es que guardar filas de una en una puede influir mucho en la latencia. Por ello, el rendimiento observado mide también el coste del registro. Hoy hemos priorizado que el registro quede completo y protegido, no optimizarlo mediante escritura por lotes.

9. Transaccion: datos, estados y motivos

Transaccion es una clase de datos. No abre conexiones y no decide si existe una colisión. Describe la operación concreta que el servidor ha recibido.

Su constructor recibe operacion, clave y valor. Genera un UUID para identificarla, pone t_final a null y empieza en INICIADA. En lecturas, valor puede ser null. “cuenta1” es la clave que se consulta o modifica; el objeto no contiene por sí mismo una transacción bancaria completa.

Los campos id, operacion, clave y valor se mantienen como final: no se reasignan durante la vida del objeto. Hoy t_inicial dejó de ser final para permitir ajustar el inicio al momento de admisión. El constructor todavía asigna una hora provisional; el control la sustituye antes de decidir.

Ya existía el enum Estado. Un enum limita un dato a una lista de valores con nombre. Hoy añadimos otro:

```
public enum MotivoAborto {
    NINGUNO, COLISION_TEMPORAL, ERROR_OPERACION
}
```

Estado responde a “¿cómo terminó?”; motivo responde a “¿por qué abortó?”. Una operación correcta tiene FINALIZADA y NINGUNO. Una colisión tiene ABORTADA y COLISION_TEMPORAL. Un fallo al ejecutar tiene ABORTADA y ERROR_OPERACION.

Los getters permiten consultar datos: getId, getOperacion, getClave, getValor, getTInicial, getTFinal, getEstado y el nuevo getMotivoAborto. Los setters actualizan los campos que cambian durante el recorrido: setTFinal, setEstado, setMotivoAborto y setTInicial.

setTInicial no lleva public: puede usarse desde las clases del mismo paquete. Esto expresa que lo ajusta el código del sistema, no que se quiera ofrecer como operación general a cualquier código externo. Los setters no implementan por sí solos todas las reglas de transición; el recorrido previsto las mantiene mediante el gestor y el control.

La fecha de la transacción no debe confundirse con las medidas de latencia del cliente. Se toman en lugares distintos y responden a preguntas distintas.

10. GestorTransacciones: campos, constructores y conexiones

El gestor conserva las referencias a Mongo7, Mongo8, Redis y el escritor del histórico. Hoy añadimos un campo ControlConcurrencia creado una sola vez por gestor. Es el punto común que consultan las peticiones concurrentes.

Hay dos constructores. El público recibe las tres bases y crea el histórico en la ruta de resultados. El otro permite pasar también un HistoricoCsv. Se utiliza en las pruebas para escribir en una carpeta temporal y no tocar los resultados reales. No son dos gestores distintos: son dos maneras de construir el mismo tipo de objeto.

La nueva ruta por defecto es resultados/historico_transacciones_concurrentes.csv. No reutilizamos el CSV secuencial de siete columnas, porque ahora hay una columna adicional. El escritor rechaza una cabecera incompatible para evitar mezclar formatos silenciosamente.

abrir(uriMongo7, uriMongo8, uriRedis) llama a open en las tres bases. Se ejecuta al iniciar el servidor, antes de entrar en el bucle de atención. No abrimos y cerramos todos los clientes de bases por cada petición.

cerrar() llama a close en las tres bases. Es una operación de cierre del servidor, no de finalización de cada transacción. El servidor intenta ejecutarla después de que terminen las tareas de atención. El cierre actual es sencillo: no constituye un protocolo de recuperación frente a cualquier fallo de infraestructura.

Mongo7 se declara con el tipo Mongo porque también utiliza guardarHistorico, que no está en DatabaseClient. Mongo8 y Redis pueden declararse con DatabaseClient porque el gestor solo necesita de ellas open, read, write y close.

No hemos añadido un nuevo acceso directo desde el cliente a las bases. Todas las decisiones de admisión y ejecución permanecen en el servidor, a través de su gestor compartido.

11. ejecutarEscritura y ejecutarLectura

Ambos métodos reciben la transacción, la proporción de lecturas del escenario y el tamaño del bloque. El porcentaje siempre representa lecturas, también cuando la operación concreta es de escritura. Es un metadato del experimento, no una instrucción para cambiar esa operación de tipo.

Primero se consulta intentarIniciar. Si devuelve false, el control ya ha marcado la colisión y los tiempos. El gestor intenta registrar el resultado y sale. La escritura retorna sin hacer operaciones. La lectura devuelve Map.of(), un mapa vacío; el servidor comprobará el estado antes de interpretar ese mapa como una lectura exitosa.

Si la transacción fue admitida, la escritura valida que su operación sea escritura y llama, en orden, a mongo7.write, mongo8.write y redis.write. Si todo termina, asigna FINALIZADA. La lectura hace lo mismo con read y guarda las respuestas en un HashMap con las claves mongo7, mongo8 y redis.

Un HashMap relaciona etiquetas y valores: por ejemplo, la etiqueta mongo7 con el texto leído allí. Es local a esa transacción. No estamos haciendo que todos los hilos escriban en un único HashMap compartido.

Si una operación lanza una excepción, el catch marca ABORTADA y ERROR_OPERACION y conserva la excepción en una variable. El finally llama siempre a control.finalizar para la transacción admitida. Después se intenta registrar el resultado y se vuelve a lanzar el error original si lo hubo.

```
Admitir -> intentar operaciones -> fijar estado
        -> liberar control -> guardar histórico -> devolver o comunicar error
```

Ejemplo de fallo parcial: Mongo7 acepta la escritura y Mongo8 falla. Redis no llega a ejecutarse, el gestor marca ABORTADA y libera el control. La escritura de Mongo7 no se revierte automáticamente. Por tanto, este sistema no garantiza atomicidad distribuida ni rollback.

En una lectura, que una clave no exista y una base devuelva null no es automáticamente una excepción. Además, todavía no se comparan los valores de las tres bases para exigir un nivel de consistencia. Esa fase se acordará más adelante con Maite.

12. registrarResultado y guardarHistorico

Estos métodos separan el trabajo de datos del trabajo de registro. Ya existían como parte del tratamiento del histórico, y hoy se adaptaron al control nuevo y al motivo de aborto.

registrarResultado obtiene inicio y fin del objeto Transaccion. No calcula otro final después de guardar el CSV. Así mantiene la frontera definida: el intervalo de la transacción termina antes de persistir el histórico.

Después llama a guardarHistorico. Si ese guardado falla y no había error de operación, propaga el error de registro. Si ya existía un error de operación, añade el de registro con addSuppressed y finalmente propaga el original.

“Excepción suprimida” no significa un error que ignoramos. Es un error adicional asociado al principal. Así no sustituimos “falló Mongo al escribir los datos” por “falló después el CSV” y perdemos la primera causa.

guardarHistorico intenta primero Mongo7 y después el CSV. Si falla Mongo, conserva el error y de todos modos intenta el archivo. Si fallan ambos, conserva también el segundo error. No hay una transacción atómica entre Mongo y el archivo: un registro podría existir en uno y faltar en otro.

Situación	Estado de la transacción	Respuesta del servidor
Datos correctos e histórico guardado	FINALIZADA	OK
Colisión e histórico guardado	ABORTADA / COLISION_TEMPORAL	ABORTADA COLISION_TEMPORAL
Fallo de datos	ABORTADA / ERROR_OPERACION	ERROR
Datos correctos, pero falla el histórico	FINALIZADA	ERROR
Colisión, pero falla el histórico	ABORTADA / COLISION_TEMPORAL	ERROR

Esto explica por qué el cliente no puede identificar todos los estados internos solo contando respuestas OK y ERROR. Un ERROR indica que la petición completa no terminó correctamente, no necesariamente que no se haya realizado ningún cambio.

En las ejecuciones reales de validación de hoy no hubo errores de registro ni de comunicación, por lo que los recuentos de estados y respuestas sí pudieron contrastarse directamente.

13. El servidor: main y la atención de conexiones

main crea un único gestor, abre un ServerSocket en el puerto 5000 y un ExecutorService con dieciséis hilos. El ServerSocket es el punto que escucha conexiones entrantes; no es una conexión a Mongo.

El bucle de escucha conserva accept(). Esta llamada espera hasta que llega una conexión y devuelve un Socket para atenderla. Antes hacíamos todo el trabajo justo ahí. Ahora entregamos una tarea al pool:

```
Socket cliente = servidor.accept();
hilos.submit(() -> atenderCliente(cliente, gestor));
```

La flecha es una lambda: una forma corta de decir “esta es la acción que ejecutarás después”. No significa que atenderCliente se ejecute inmediatamente en el hilo principal. submit entrega el encargo al pool y permite que el bucle vuelva a aceptar conexiones.

El pool tiene un límite de dieciséis trabajadores. Las tareas adicionales pueden quedar en su cola. Esa cola no es el protocolo de aborto: una petición que todavía no ha llegado al gestor no ocupa activa. El pool fijo utilizado tiene una cola que no hemos acotado específicamente; no se ha implementado una política completa de rechazo por sobrecarga.

atenderCliente se ejecuta en un trabajador. Mantiene el socket dentro de try-with-resources, fija un límite de cinco segundos para esperar la línea, crea un lector y un escritor UTF-8, llama a procesarPeticion y envía una respuesta. Una conexión que no envía nada no puede retener indefinidamente un hilo en esa lectura.

El límite de cinco segundos es para leer el mensaje del socket. No es un plazo general para terminar cualquier operación de base de datos. La latencia de las bases y sus propios límites de espera son otra cuestión.

PrintWriter se crea con autoflush: al enviar la línea de respuesta, se fuerza su salida. Si la conexión falla, se registra el problema. No hay un mecanismo de entrega garantizada o reintento de respuestas perdidas.

14. El servidor: procesarPeticion y el cierre

procesarPeticion conserva la validación del protocolo. Divide la línea en hasta cinco partes, para que el valor de una escritura pueda contener espacios. Convierte READ o WRITE a mayúsculas y comprueba el número de argumentos.

Convierte la proporción a double y el bloque a int. Rechaza valores no numéricos, una proporción fuera de cero a uno, NaN o infinito y bloques no positivos. Una petición inválida responde ERROR antes de crear o ejecutar una transacción; no se registra como una colisión.

Ejemplos válidos:

```
READ 0.5 32 cuenta1
WRITE 0.5 32 cuenta1 valor_7
```

Después crea Transaccion y llama al gestor. El cambio importante de hoy es que conserva ese objeto para consultar su estado al volver. Si está ABORTADA, devuelve ABORTADA seguido del motivo. Si la llamada lanzó una excepción, entra en el catch y devuelve ERROR. Si todo fue correcto, devuelve OK READ con los valores o OK WRITE con la clave.

Una respuesta de lectura elimina saltos de línea de los valores para no romper el protocolo de una sola línea. El cliente usa readLine y espera precisamente una línea de respuesta.

Cierre del servidor. Hemos añadido un shutdown hook, una acción que la JVM intenta ejecutar al cerrarse normalmente, por ejemplo al recibir Ctrl+C. Cierra el ServerSocket para desbloquear accept y espera al hilo principal hasta 150.000 milisegundos. El cierre del ExecutorService, declarado como recurso en Java 21, espera a las tareas; después se intenta cerrar el gestor.

Thread.currentThread obtiene el hilo actual. join espera a que otro hilo termine. Si una espera es interrumpida, se repone la señal mediante interrupt. Al salir normalmente se intenta retirar el hook; si la JVM ya está cerrando, esa retirada puede lanzar IllegalStateException y se trata como parte del cierre.

No debemos interpretar esto como garantía ante un apagado forzado, una caída del proceso o una tarea que no termine nunca. Es un cierre ordenado básico para nuestro servidor experimental. No hemos probado todas las situaciones de fallo del sistema operativo.

15. El cliente: main, Envio y prepararOperaciones

main lee un argumento opcional: la concurrencia. Si no hay ninguno, usa cuatro. Integer.parseInt convierte el texto recibido en un entero. Se comprueba que esté entre uno y sesenta y cuatro antes de crear el CSV. Un argumento no numérico o fuera de rango impide iniciar los experimentos.

Crea ResumenExperimentosCsv dentro de try-with-resources, muestra su ruta y llama a ejecutarExperimentos con la concurrencia elegida. try-with-resources significa que el recurso se cerrará al salir del bloque, también si ocurre una excepción.

Envio es una interfaz pequeña con un método enviar(String comando). Permite separar la preparación del experimento del mecanismo de envío. En una ejecución real se conecta al servidor; en una prueba puede devolver una respuesta simulada. Esa posibilidad ya existía y permite comprobar 5.110 intentos sin necesitar bases reales en cada prueba.

La lambda del main conecta esa interfaz con el método de red:

```
comando -> enviarPeticion("localhost", 5000, comando)
```

Se lee: “cuando te entreguen un comando, envíalo a localhost, puerto 5000”. No genera código oculto ni cambia el protocolo.

prepararOperaciones calcula cuántas lecturas corresponden al bloque con Math.round, crea una lista de READ y WRITE y la mezcla con Collections.shuffle. Para un bloque de dos y proporción 0,2, se redondea 0,4 a cero lecturas. Por eso el porcentaje real no siempre coincide con el nominal en los bloques pequeños.

WORKLOADS, BLOQUES y CLAVES siguen definidos en la clase. No se han convertido hoy en argumentos configurables. Solo la concurrencia se elige desde la línea de comandos. Se mantienen cinco mezclas, nueve bloques y cuatro claves.

Random se utiliza en la preparación, desde el hilo que organiza los experimentos. Los comandos quedan preparados antes de medir el bloque y antes de entregarlos a los trabajadores. No se ha fijado una semilla para reproducir exactamente el orden de operaciones entre ejecuciones.

16. ejecutarExperimentos: tres firmas y un recorrido

Verás tres métodos llamados ejecutarExperimentos. Es una sobrecarga: comparten nombre pero reciben parámetros distintos. Las variantes cortas existen para las pruebas anteriores y delegan en la completa con concurrencia uno. La variante usada por main recibe Envio, PrintStream, ResumenExperimentosCsv y concurrencia.

PrintStream representa dónde se escribe la salida, normalmente System.out. En las pruebas puede ser una salida en memoria o una salida descartada. ResumenExperimentosCsv puede ser null en las pruebas que no necesitan archivos; antes de guardar se comprueba que exista.

La variante completa crea un pool con el número de hilos elegido. Recorre las mezclas y, dentro de cada una, los bloques. Prepara todos los comandos del bloque: tipo, proporción, tamaño, clave y, si corresponde, valor.

Después pone los contadores a cero, crea una lista de Future y un CountDownLatch de valor uno. El latch funciona como una puerta de salida: las tareas esperan hasta que el organizador termina de entregarlas y abre la puerta con countDown.

Esto no obliga a que todos los hilos ejecuten una instrucción en el mismo nanosegundo. Da un punto común de liberación, pero la planificación posterior depende de la JVM y del sistema operativo. Además, si hay más tareas que hilos, algunas seguirán esperando en el pool.

El inicio del tiempo de bloque se mide antes de enviar las tareas a submit. Una vez entregadas, se abre el latch. Solo entonces se comienza a recoger los resultados con Future.get.

Una duda habitual: si get espera, ¿volvemos a lo secuencial? No. Ya se enviaron todas las tareas y los trabajadores siguen ejecutándolas. El hilo principal puede estar esperando el resultado de T1 mientras T2, T3 y T4 ya están trabajando o incluso han terminado.

Cuando ha recogido todos los resultados, calcula la duración del bloque. Guarda una fila de resumen y muestra los valores en consola. Solo después empieza otro bloque. Así no se superponen escenarios distintos dentro de esta ejecución del cliente.

17. ResultadoPeticion, ejecutarPeticion y enviarPeticion

ResultadoPeticion es una clase pequeña dentro del cliente. Cada tarea crea su propio objeto con cuatro datos: duracionNs, respuesta, correcta y colision. No hay una clase de producción adicional en otro archivo para algo que solo usa este cliente.

ejecutarPeticion mide el tiempo de un intento y llama a Envio.enviar. Considera correcta una respuesta que comienza por OK y por el tipo de operación esperado. Considera colisión exactamente la respuesta ABORTADA COLISION_TEMPORAL. Una respuesta distinta o una IOException quedan como otro error.

Al volver, la tarea devuelve su ResultadoPeticion. Los trabajadores no incrementan directamente correctas, abortadas o errores. El hilo principal obtiene los resultados y actualiza esos contadores. Así evitamos perder incrementos por escrituras simultáneas en variables compartidas.

enviarPeticion realiza la comunicación concreta. Crea un Socket, conecta a la dirección indicada con límite de cinco segundos y fija un límite de 120 segundos para esperar la lectura de la respuesta. Crea un BufferedWriter y un BufferedReader con codificación UTF-8.

Escribe el comando, añade un salto de línea y llama a flush. El salto permite que readLine del servidor sepa dónde termina el mensaje. flush fuerza a enviar lo que pudiera quedar en el búfer. Después el cliente lee una línea.

Si readLine devuelve null, el servidor cerró la conexión sin entregar respuesta. Se lanza EOFException, que es un tipo de IOException. Los recursos se cierran automáticamente al salir de sus bloques.

Las conexiones son independientes: cada petición abre su propio socket. No se ha diseñado hoy un protocolo que envíe muchas peticiones por una misma conexión persistente.

InterruptedException indica que una espera ha sido interrumpida. Se pide detener el pool, se repone la señal de interrupción y se comunica que el experimento puede quedar incompleto. **ExecutionException** envuelve un error inesperado dentro de una tarea: se comunica su causa y se interrumpe la batería. shutdownNow solicita interrupción; no garantiza que cualquier operación externa termine instantáneamente.

18. Bibliotecas y expresiones de Java que aparecen

No se añadieron dependencias externas en pom.xml para la concurrencia. Las herramientas siguientes forman parte de Java. Los imports indican dónde encontrar una clase; no descargan una biblioteca por sí mismos.

Herramienta	Dónde y para qué se usa
ExecutorService / Executors	Pools del cliente y del servidor; ejecutar tareas
Future	Representa un resultado que puede no estar listo todavía
Callable	Tipo de tarea que devuelve un resultado; aparece a través de la lambda de submit
CountDownLatch	Puerta de salida del bloque y coordinación de las pruebas
ReentrantLock	Protección explícita de la admisión y finalización
Thread / Runtime	Hilo actual, interrupción y cierre del servidor
Socket / ServerSocket	Conexión individual y escucha de conexiones
BufferedReader / BufferedWriter	Lectura y escritura de texto con búfer
Files / Path	Rutas, creación de carpetas y acceso a archivos
Timestamp / System	Fechas del histórico y relojes para las medidas
List, ArrayList, Map, HashMap	Colecciones para comandos, tareas y respuestas

Las herramientas de concurrencia están principalmente en java.util.concurrent. ReentrantLock está en su subpaquete locks. synchronized es una palabra del lenguaje, no una biblioteca que haya que instalar.

En las pruebas también usamos AtomicInteger para contar desde distintos hilos, ConcurrentHashMap.newKeySet para reunir nombres de hilos sin perder datos y TimeUnit para expresar límites de espera. No trasladamos esos contadores compartidos al cliente real porque este recoge resultados en un único hilo.

private limita el acceso a la clase; **public** permite acceso desde fuera; **static** indica que el miembro pertenece a la clase y no exige un objeto particular; **final** evita reasignar una referencia. Ninguna de esas palabras equivale por sí sola a “seguro con hilos”.

List<Future<ResultadoPeticion>> se puede leer de dentro hacia fuera: “resultado de una petición”, “promesa de obtenerlo después”, “lista de esas promesas”. Los símbolos no introducen un mecanismo diferente al que hemos explicado.

try, catch y finally significan intentar, tratar un error y realizar una acción de salida. **throws** anuncia que el método puede comunicar una excepción a quien lo llamó. **throw** la lanza en un punto concreto.

19. Mongo, Redis y DatabaseClient

DatabaseClient conserva cuatro métodos: open(String uri), write(String key, String value), read(String key) y close(). Es el contrato común para hablar con distintos tipos de base. No añadimos el control de colisiones a esta interfaz porque es una decisión del gestor, no de cada base por separado.

Mongo.open interpreta la URI, exige un nombre de base y crea el cliente y las referencias a mensajes e historico. **write** sustituye o inserta el documento de la clave mediante replaceOne y upsert. **read** busca por _id y devuelve mensaje, o null si no existe. **close** cierra el cliente. Estos métodos ya existían.

El cambio de hoy en Mongo está en **guardarHistorico**: recibe MotivoAborto y lo almacena con el nombre motivoAborto, junto al estado y los tiempos. El método name() del enum obtiene un texto como COLISION_TEMPORAL. Los documentos antiguos no se migran ni se eliminan.

La dependencia mongodb-driver-sync 4.11.1 ya estaba en pom.xml. Es el controlador utilizado por las llamadas síncronas de Mongo. En este diseño se comparte el cliente y las referencias a colecciones; cada llamada construye su documento y consulta de forma local. Las operaciones de cuentas siguen admitidas de una en una, aunque los guardados de históricos puedan coincidir.

Redis no necesitó cambios hoy. Usa Jedis 5.1.0 y un JedisPool. open crea el pool; write obtiene una conexión, hace set y la devuelve al salir del try; read hace lo mismo con get. close cierra el pool. No se comparte una única instancia de Jedis para todas las tareas.

Pool de conexiones y pool de hilos son conceptos distintos: el primero presta conexiones a una base; el segundo presta trabajadores para ejecutar tareas. Que ambos se llamen pool no significa que tengan el mismo tamaño o que una petición posea siempre un recurso de cada pool.

Las tres bases están en contenedores locales con puertos diferentes. Dos instancias de Mongo no significan dos implementaciones de la clase Mongo. Redis y Mongo comparten la interfaz, pero no el protocolo interno de sus controladores.

20. HistoricoCsv: guardar filas sin mezclarlas

HistoricoCsv recibe una Path en su constructor y conserva esa ruta. Su método guardar recibe los datos de una transacción. Hoy añadimos el motivo y la palabra synchronized a ese método.

synchronized hace que dos hilos que usan la misma instancia no ejecuten el método a la vez. El servidor comparte un solo escritor a través del gestor. Si se creasen dos escritores independientes para el mismo archivo, o dos procesos de servidor, esta protección no los coordinaría. No es un bloqueo del archivo a nivel distribuido.

Dentro de guardar se crea la carpeta si hace falta. Se comprueba si el archivo está vacío o no existe. Si ya tiene contenido, se lee su primera línea y se exige que la cabecera sea la esperada. Se abre con CREATE y APPEND: se crea si falta y se añade al final si existe.

La primera escritura añade la cabecera. Cada escritura añade una fila y un salto de línea. El identificador se escapa entre comillas, duplicando las comillas interiores, para mantener un CSV válido. Los números se escriben con punto decimal. El motivo y el estado proceden de enums con nombres controlados.

Las ocho columnas son:

```
id, tiempoInicio, tiempoFinal, duracionMs,
porcentajeLectura, bloque, estado, motivoAborto
```

El campo duracionMs es fin menos inicio del gestor. No incluye guardar el propio histórico ni viajar de vuelta al cliente. Una colisión puede registrar cero milisegundos porque se decidió dentro del mismo milisegundo.

El archivo acumula transacciones de varias ejecuciones. Los resúmenes se separan por archivo, pero el histórico actual no tiene un identificador de ejecución. Para las comprobaciones de hoy utilizamos que el archivo concurrente era nuevo y conocíamos el orden de las dos ejecuciones. Esto es una limitación de trazabilidad que conviene recordar para experimentos futuros.

El histórico no contiene actualmente la clave ni el tipo de operación, aunque el objeto Transaccion sí los tenga. Hoy no añadimos esas columnas. Tampoco guarda peticiones que se rechazan por formato antes de construir una transacción.

21. ResumenExperimentosCsv y sus quince columnas

El constructor crea la carpeta y un nombre nuevo con fecha e identificador: resumen_concurrente_FECHA_IDENTIFICADOR.csv. Files.createTempFile se utiliza para obtener un nombre único dentro de resultados; no significa que el archivo se borre automáticamente al terminar.

El constructor abre el escritor, añade la cabecera y hace flush. Si falla esa preparación, intenta cerrar el recurso y propaga el error. **getRuta** permite al cliente mostrar el archivo creado. **guardar** añade una fila por bloque. **close** cierra el escritor al salir del try-with-resources.

Las doce primeras columnas conservan el orden anterior. Añadimos tres al final:

Columna	Significado
proporcionLecturas	Mezcla nominal entre cero y uno
proporcionEscrituras	Uno menos la proporción de lecturas
bloque	Total de peticiones del bloque
lecturas / escrituras	Cantidades realmente preparadas y enviadas
correctas	Respuestas OK recibidas
fallidas	Colisiones más otros errores
tiempoTotalNs	Duración del bloque en nanosegundos
tiempoPeticionesNs	Suma de duraciones de todos los intentos
tiempoTotalMs	Duración del bloque convertida a milisegundos
latenciaMediaMs	Media por intento, incluidos abortos y errores
throughputPeticionesSegundo	Respuestas correctas por segundo de bloque
concurrencia	Número configurado de trabajadores del cliente
abortadasColision	Respuestas ABORTADA COLISION_TEMPORAL
errores	Otras respuestas o errores de comunicación

Son quince columnas: lecturas y escrituras ocupan dos posiciones distintas. El escritor se llama desde el hilo principal del cliente, después de recoger todos los Future; por eso no necesita el mismo synchronized que el histórico del servidor.

Si la batería termina, hay 45 filas de datos más una cabecera. Si se interrumpe, puede haber menos. Cada archivo representa una ejecución y una concurrencia configurada. No se sobrescriben los CSV secuenciales ni los resúmenes anteriores.

22. Entender las medidas y no sacar conclusiones falsas

System.currentTimeMillis da una marca de calendario útil para registrar cuándo ocurrió algo. System.nanoTime se usa para medir duraciones mediante diferencias: no es una fecha para mostrar en Excel. Los nanosegundos son la unidad del valor, no una promesa de exactitud física de un nanosegundo.

Las fórmulas son:

```
tiempoTotalMs = tiempoTotalNs / 1.000.000
latenciaMediaMs = tiempoPeticionesNs / 1.000.000 / bloque
throughput = correctas * 1.000.000.000 / tiempoTotalNs
fallidas = abortadasColision + errores
bloque = correctas + abortadasColision + errores
```

La latencia empieza dentro de ejecutarPeticion, antes de conectar, y acaba después de recibir respuesta o error. Incluye comunicación, esperas en el servidor y guardado del histórico antes de responder. No incluye la espera previa de esa tarea en la cola del cliente ni en el latch de salida.

El tiempo total del bloque empieza antes de entregar tareas al pool y termina tras recoger todos los resultados. Incluye esa organización y espera global. Excluye preparar los comandos, guardar el resumen y mostrar el resumen en consola. Los mensajes de errores que se imprimen al recoger resultados sí pueden influir en el intervalo medido.

En concurrencia, sumar las duraciones individuales puede dar más que el tiempo total. Ejemplo didáctico: dos peticiones duran diez milisegundos cada una y se solapan por completo. Su suma es veinte, pero el tiempo transcurrido puede rondar diez. No es un error de unidades.

El throughput cuenta solo las correctas, no las abortadas. Una latencia baja puede deberse a rechazar muy rápido muchas peticiones. Por eso se deben mirar juntas latencia, throughput y proporción de abortos.

El parámetro concurrencia es un máximo de tareas en curso, no una garantía de igual solapamiento en todos los bloques. Los pequeños tienen menos tareas; las bases, la planificación y el histórico influyen en cuánto coinciden. Un pool de cuatro no implica que tres de cada cuatro peticiones aborten siempre.

23. Las pruebas automáticas: cómo comprobamos la regla

Ejecutamos mvn -o test. El modificador -o utiliza las dependencias ya disponibles localmente; no cambia lo que significan las pruebas. Pasaron 28 pruebas, sin fallos ni errores.

Clase de pruebas	Cantidad	Qué cubre
ConcurrenciaTest	5	Admisión, sockets, errores, CSV y cliente concurrente
EstadosTransaccionTest	5	Estados, tiempos y fallos del histórico
ExperimentosTest	4	Batería, protocolo, errores y sockets
HistoricoCsvTest	4	Filas, cabecera y los dos destinos de registro
MongoTest	5	Validación y propagación de errores del controlador
RedisTest	3	Configuración y tratamiento de errores
ResumenExperimentosCsvTest	2	Resúmenes, fórmulas, nombres nuevos y fallos de escritura

Las pruebas no necesitan Docker. Usan objetos simulados que devuelven valores o errores conocidos. Eso no sustituye probar las bases reales, pero permite provocar exactamente los casos que queremos estudiar.

ConcurrenciaTest tiene setUp y tearDown: antes de cada prueba crea una carpeta temporal y después elimina sus archivos. La clase interna Base simula Mongo. Cuenta operaciones e históricos con AtomicInteger y tiene dos CountDownLatch para indicar “ya estoy dentro” y “puedes continuar”.

testSoloUnaAdmitidaAunqueLleguenALaVez: lanza treinta y dos intentos con ocho trabajadores y mantiene ocupada la primera admisión. Comprueba una admitida, treinta y una colisiones y que, después de liberar, otra puede entrar. No pretende que los treinta y dos estén físicamente ejecutándose a la vez.

testColisionPorSocketLecturasYEscriturasDistintasClaves: usa sockets locales y el método real atenderCliente. Mantiene una lectura en curso, envía otra lectura y una escritura con claves diferentes y verifica los abortos. Comprueba que las rechazadas no incrementaron el contador de operaciones de datos. Libera la primera y verifica que una nueva escritura sí funciona.

24. Más pruebas y resultados reales de hoy

testErrorLiberaControl: fuerza un fallo de operación, comprueba ERROR_OPERACION y verifica que una transacción posterior puede finalizar. Detectaría un control que quedara ocupado para siempre tras una excepción.

testCsvNoMezclaFilasConcurrentes: solicita cien guardados desde ocho hilos. Espera cien filas más cabecera, identificadores únicos y ocho columnas por fila. Busca errores de escritura compartida, no rendimiento del disco.

testClienteConcurrenteYRecuentosCsv: obliga a que los dos primeros envíos simulados estén en curso antes de continuar. Luego alterna respuestas correctas, colisiones y errores de comunicación. Comprueba 5.110 intentos, 45 resúmenes y recuentos conocidos. Los resultados simulados son 1.703 correctas, 1.703 colisiones y 1.704 errores; no deben confundirse con las ejecuciones reales.

Las pruebas previas de Mongo, estados e histórico se adaptaron al nuevo parámetro motivo. Se conservó su comprobación de que los errores no se oculten.

Después arrancamos el servidor concurrente con las bases reales. Docker inicialmente devolvió un error interno; se reanudó la validación cuando los contenedores estuvieron disponibles. No se presentaron como resultados válidos intentos contra bases inaccesibles.

Ejecución real	Resúmenes	Intentos	Correctas	Colisiones	Otros errores
Concurrencia 4	45	5.110	4.974	136	0
Concurrencia 1	45	5.110	5.110	0	0

Se contrastaron los 10.220 registros del CSV con Mongo por identificador y campos: tiempos, duración, mezcla, bloque, estado y motivo. También se comprobaron recuentos por escenario. El conjunto contiene 10.084 FINALIZADA/NINGUNO y 136 ABORTADA/COLISION_TEMPORAL.

En la primera ejecución se revisó además que los intervalos guardados fueran compatibles con la regla, dentro de la precisión de milisegundos, y que las fórmulas del resumen coincidieran con los tiempos. La prueba controlada con hilos aporta evidencia que los tiempos redondeados por sí solos no pueden ofrecer.

Estos resultados validan los casos ejercitados. No prueban ausencia absoluta de errores ni que la versión sea más rápida en cualquier equipo o carga. Son una ejecución con cuatro envíos y otra con uno, sin una campaña de repeticiones ni control estadístico.

25. Cómo ejecutar y encontrar los resultados

Antes de ejecutar, abre Docker Desktop y comprueba mongo7, mongo8 y redis. El servidor usa Mongo7 en 27017, Mongo8 en 27018 y Redis en 6379. Ejecuta desde la raíz del proyecto para que las rutas relativas apunten a la carpeta esperada.

Primero comprueba dónde estás trabajando:

```
git branch --show-current
git status
```

La rama debe ser version-concurrente. No inicies dos servidores en el puerto 5000. Si había un servidor de la versión anterior, ciérralo antes de arrancar el nuevo para que cargue las clases actuales.

En una terminal:

```
mvn -Pservidor compile exec:java
```

En otra, para cuatro envíos simultáneos:

```
mvn -Pcliente compile exec:java "-Dexec.args=4"
```

Puedes cambiar cuatro por uno u ocho. El intervalo admitido es de uno a sesenta y cuatro. Los dieciséis trabajadores del servidor están definidos en su código y no se cambian con ese argumento del cliente.

Al iniciar, el cliente imprime la ruta del resumen. Al terminar, busca 45 filas y revisa las columnas correctas, abortadasColision y errores. Verifica que sumen el bloque. Si hay colisiones, no borres esas filas: son parte del experimento. Si hay otros errores, hay que investigar antes de usar los datos como resultados de rendimiento.

El histórico está en resultados/historico_transacciones_concurrentes.csv. Acumula las ejecuciones. Los resúmenes usan nombres nuevos. Mongo guarda el histórico en tfg_mongo7, colección historico. Los archivos anteriores de la versión secuencial siguen conservados.

Para ejecutar las pruebas sin bases, usa mvn test. Los archivos de resultados no se suben a Git porque resultados está excluida. Crear una rama o guardar un archivo tampoco lo sube automáticamente: commit y push son pasos separados.

26. Los archivos de esta validación y Excel

Los archivos siguientes están dentro de resultados, en la raíz del proyecto:

```
resumen_concurrente_20261012_131616_6949642962257001128.csv
resumen_concurrente_20261012_131829_4205797332519414778.csv
historico_transacciones_concurrentes.csv
validacion_concurrente_resumen.txt
```

El primer resumen es la ejecución con cuatro envíos. El segundo es la de un envío. Se guardaron también logs del cliente y servidor y una exportación JSON de Mongo para contrastar datos. No son requisitos para que el programa arranque; son evidencia auxiliar de estas comprobaciones.

Para importar en Excel, usa Datos > Desde texto/CSV, delimitador coma y UTF-8. En Power Query, si la conversión automática interpreta mal los puntos, elimina ese paso y convierte desde los valores originales usando Número decimal y configuración regional Inglés (Estados Unidos). No basta con cambiar a decimal después de haber perdido las cifras.

Las proporciones deben conservar 0, 0,2, 0,5, 0,8 y 1. Un valor como 0.19999999999999996 puede aparecer al calcular 1 - 0.8 con double; se muestra como 20 % para facilitar la lectura, sin inventar otro escenario.

Si haces tablas dinámicas, usa bloque en filas y proporcionLecturas en columnas. Elige la métrica correspondiente en valores. Cada combinación aparece una vez por resumen; Promedio no significa que ya hayamos repetido el experimento varias veces.

No incluyas Total general como un punto después de 512. Ese fue el problema detectado en el Excel secuencial: mezclaba un agregado con los tamaños de bloque. Etiqueta ambos ejes y sus unidades. Guarda las gráficas en .xlsx, porque CSV solo conserva la tabla de texto.

Para la versión concurrente hay que acompañar latencia y throughput con los abortos. No mezcles dos archivos de concurrencias distintas sin añadir esa dimensión. En este trabajo todavía no se han creado gráficas concurrentes definitivas ni conclusiones estadísticas.

27. Cómo leer el código para poder explicarlo

No empieces intentando memorizar todos los imports. Abre primero ControlConcurrencia y sigue una transacción admitida y otra rechazada. Después busca las dos llamadas a ese control en GestorTransacciones. Cuando entiendas eso, pasa al servidor y al cliente.

Un orden de estudio útil es: Transaccion, ControlConcurrencia, GestorTransacciones, ServidorTransaccionesSimple, ClienteTransaccionesSimple, HistoricoCsv, ResumenExperimentosCsv y ConcurrenciaTest. Mongo y Redis se pueden revisar después para recordar cómo se hacen las operaciones concretas.

Pregunta 1. ¿Por qué hay dos pools? Porque el cliente necesita realizar envíos concurrentes y el servidor necesita atender conexiones concurrentes. Son programas separados y sus tareas no son las mismas.

Pregunta 2. ¿Por qué no ponemos un lock alrededor de toda la escritura? Porque las demás peticiones podrían esperar a que acabara y ejecutarse después sin colisionar. El diseño protege la decisión breve y conserva la referencia activa durante las operaciones.

Pregunta 3. ¿Por qué una lectura puede abortar si no modifica datos? Porque la regla de esta fase rechaza cualquier solapamiento, no solo los conflictos entre escrituras.

Pregunta 4. ¿Por qué Future.get no hace secuencial el experimento? Porque todas las tareas del bloque se entregaron antes al pool. Mientras el hilo principal espera un resultado, los trabajadores siguen activos.

Pregunta 5. ¿Por qué son necesarias dos clases CSV? Una registra cada transacción en el servidor; otra resume cada combinación experimental en el cliente. Tienen distinta granularidad y miden tiempos distintos.

Pregunta 6. ¿Por qué puede haber FINALIZADA con respuesta ERROR? Porque las operaciones terminaron, pero falló el registro posterior. Estado de las operaciones y éxito de toda la petición no son idénticos.

Pregunta 7. ¿Hemos implementado ya el autómata completo de Pablo? No. Hemos usado su trabajo como referencia, pero esta fase aplica la regla temporal simplificada de Maite y deja los niveles de consistencia para después.

28. Límites actuales y relación con los TFG de referencia

El TFG de Conrad sigue siendo la referencia general del proyecto. El TFG de Pablo se utilizó hoy para estudiar concurrencia, validación, registro y el autómata de su página numerada 35, que es la página 36 del PDF proporcionado.

En la sección de validación, Pablo describe condiciones relacionadas con escrituras, cuentas y tiempos. No hemos trasladado esas condiciones al programa: la instrucción confirmada de Maite exige abortar ante cualquier solapamiento. La diferencia debe quedar clara al explicar el trabajo y al redactar la memoria.

El autómata de Pablo incluye también decisiones de consistencia de lectura y escritura. Esta versión no tiene parámetros R y W ni selección de cuántas réplicas bastan, ni una comprobación completa de igualdad de lecturas. Tampoco hemos supuesto cuál es el detalle del dibujo que Maite dijo que corregiría.

Otros límites que conviene poder explicar: el control es global y local a un servidor; no hay rollback; no hay reintentos automáticos; no existe un protocolo de idempotencia para peticiones repetidas; un error de comunicación no confirma el resultado real; el histórico puede fallar y quedar incompleto en un destino; el pool del servidor no incorpora una política completa de sobrecarga.

Los tiempos de calendario del histórico pueden verse afectados por ajustes del reloj y tienen resolución de milisegundos. Las duraciones del cliente se miden con nanoTime. Los ensayos utilizan solo cuatro claves y no tienen semilla fija ni repeticiones estadísticas. Los datos de hoy no justifican afirmar que se hayan resuelto todas las anomalías de una base distribuida.

Nada de esto invalida el objetivo de esta fase: construir una primera versión concurrente, aplicar el rechazo temporal de forma coordinada y comprobar su comportamiento. Sí delimita lo que podemos afirmar ante Maite.

29. Guion de explicación y próximos pasos

Una forma de explicarlo con tus palabras sería:

“Partí de la versión secuencial y creé otra rama. El cliente ahora envía varias peticiones con un pool de hilos, y el servidor las atiende con otro. Todas pasan por el mismo gestor. Añadí una clase que protege con un lock la comprobación de si hay una transacción activa. Si la hay, la nueva aborta por colisión temporal antes de operar sobre los datos. Si no, entra y libera su posición al terminar, incluso si falla. Registro el estado y el motivo en Mongo y CSV, y el cliente distingue colisiones de otros errores. Lo he probado con casos controlados y con dos ejecuciones reales.”

No hace falta repetir ese texto de memoria. Debes poder señalar en el código dónde sucede cada cosa: intentarIniciar para la admisión, finally para la liberación, submit para entregar tareas y get para recoger resultados.

Antes de avanzar conviene revisar con Maite la frontera de inicio y fin que se ha definido, y enseñar los recuentos de éxito y colisión. También conviene decidir cómo se organizará la próxima campaña de experimentos: concurrencias a probar, repeticiones y forma de identificar ejecuciones en el histórico si se necesita más trazabilidad.

El paso de Git pendiente es revisar el diff, hacer commit y subir la rama cuando se decida. No hay que fusionar automáticamente esta versión con main ni borrar la secuencial. El nuevo desarrollo se conserva separado para poder comparar y volver al estado validado.

La siguiente fase funcional será la que indique Maite sobre niveles de consistencia. No se añadió de forma anticipada. El orden de trabajo sigue siendo entender, implementar, comprobar y después ampliar.

30. Glosario y fuentes de esta guía

Término	Significado en este proyecto
Admisión	Decisión de permitir operaciones a una transacción
activa	Referencia a la única transacción admitida en el gestor
Aborto temporal	Rechazo porque otra transacción ocupa el intervalo
Bloque	Número total de peticiones de una combinación experimental
Concurrencia	Máximo configurado de tareas de envío del cliente
Histórico	Registro individual de cada transacción que llega al gestor
Resumen	Una fila con resultados agregados de un bloque
Pool	Grupo reutilizable de hilos o de conexiones, según el contexto
Future	Objeto con el que se obtiene el resultado de una tarea
Latch	Puerta que se abre cuando su contador llega a cero
Lock	Protección explícita de una sección de código
synchronized	Protección de Java asociada aquí a la instancia del escritor
Throughput	Respuestas correctas por segundo de tiempo del bloque
Latencia	Duración de un intento vista desde el cliente
Rollback	Deshacer cambios; no está implementado entre las tres bases
Consistencia	Reglas sobre las lecturas y escrituras que se abordarán después

La fuente principal es el código local de la rama version-concurrente leído el 12/10/2026: las clases explicadas, DatabaseClient.java, Redis.java, pom.xml, las pruebas y README.md. Los recuentos reales proceden de los CSV y del informe validacion_concurrente_resumen.txt, contrastados durante la sesión.

Como contexto se consultó el documento de Pablo García Rodríguez, “Transacciones y gestión de datos en un clúster de BBDD NoSQL homogéneas”, julio de 2025, especialmente las secciones 3.3, 3.5, 3.6 y 3.8. Las decisiones concretas implementadas responden a las indicaciones de Maite confirmadas en la conversación.

Los fragmentos de código del manual son extractos o esquemas señalados como tales. El código completo y ejecutable sigue estando en src. La guía no modifica los programas ni reemplaza los resultados originales.